

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY  
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY  
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**REPORT**

---

**STREAMING LAKEHOUSE DATA PLATFORM**

---

**COURSE: DISTRIBUTED SYSTEMS**

**SUPERVISOR(s):** PhD. NGUYEN QUANG HUNG

——o0o——

**STUDENT 1:** NGO QUANG HIEU - 2311015

**STUDENT 2:** <NAME> - 2311016

**STUDENT 3:** <NAME> - 2311017

**HO CHI MINH CITY, Sep 2026**

# Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of PhD. Diep Thanh Dang at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during my research.

Ho Chi Minh City, December 2027

*Authors,*

*Ngo Quang Hieu*

# Acknowledgement

We would like to express my appreciation to Dr. Diep Thanh Dang for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

Besides, we are grateful for the help from ... for valuable comments for research aspect and our proposed pipeline, including pointing out strengths as well as weaknesses of previous works on multimodal models so that we can elaborate on our ideas more effectively.

In addition, we would like to extend our gratitude to my family. Their unwavering faith in our abilities and constant encouragement have been my pillars of strength. Their belief in my potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

# Abstract

This thesis presents *Streamify*, an end-to-end streaming lakehouse pipeline built to demonstrate real-time data engineering under a small, fixed compute budget. Synthetic music-streaming events, generated by Eventsim against the Million Song Dataset, are ingested through a 2-node Apache Kafka (KRaft) cluster, consumed by an Apache Spark Structured Streaming job, and landed as MERGE\_ON\_READ Apache Hudi bronze tables on MinIO/S3-compatible object storage.

dbt then transforms the bronze layer, through the Spark Thrift Server, into a COPY\_ON\_WRITE star schema (a fact\_streams table and five dimension tables) and a denormalized wide\_streams table, which Apache Superset visualizes through interactive dashboards. A Prometheus/Grafana/cAdvisor stack monitors Kafka consumer lag and Spark resource usage throughout, and the entire system is deployed reproducibly with a single Docker Compose command.

Every major technology choice in the pipeline – Kafka over HTTP or RabbitMQ, Spark Structured Streaming over Flink, Hudi over plain Parquet or a managed warehouse, dbt over hand-written SQL scripts, and Spark Standalone over YARN or Kubernetes – is justified against concrete alternatives and the resource constraints of a single-machine, 4-core deployment, rather than assumed as a given. This thesis documents that reasoning alongside the resulting architecture and its observed operational behavior.

# Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                               | <b>1</b>  |
| 1.1      | Motivation . . . . .                                              | 1         |
| 1.2      | Goals . . . . .                                                   | 2         |
| 1.3      | Scope . . . . .                                                   | 3         |
| 1.4      | Thesis Structure . . . . .                                        | 3         |
| <b>2</b> | <b>Related Work and Background</b>                                | <b>5</b>  |
| 2.1      | Why Real-Time Processing Matters . . . . .                        | 5         |
| 2.2      | Message Queues and Publish-Subscribe Messaging . . . . .          | 6         |
| 2.3      | Stream Processing Engines . . . . .                               | 7         |
| 2.4      | Storage layer & Open Table Formats . . . . .                      | 8         |
| 2.5      | Medallion Layering . . . . .                                      | 9         |
| <b>3</b> | <b>Problem Statement and Challenges</b>                           | <b>10</b> |
| 3.1      | Dataset . . . . .                                                 | 10        |
| 3.2      | Technical Challenges . . . . .                                    | 10        |
| 3.2.1    | Streaming Volume and Velocity . . . . .                           | 10        |
| 3.2.2    | Exactly-once, Fault-tolerant Processing on a 2-node Kafka Cluster | 11        |
| 3.2.3    | Computing and Partitioning on a Small, Shared Spark Cluster .     | 11        |
| 3.2.4    | Dirty Keys Breaking Dimensional Joins . . . . .                   | 11        |
| <b>4</b> | <b>Proposed Solution</b>                                          | <b>13</b> |
| 4.1      | Architecture Overview . . . . .                                   | 13        |
| 4.2      | Ingestion Layer . . . . .                                         | 15        |
| 4.3      | Stream Processing Layer . . . . .                                 | 16        |

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| 4.4      | Storage Layer . . . . .                      | 19        |
| 4.5      | Transformation Layer . . . . .               | 20        |
| 4.6      | Serving Layer . . . . .                      | 21        |
| 4.7      | Deployment & Environment . . . . .           | 22        |
| 4.8      | Monitoring as Qualitative Evidence . . . . . | 23        |
| <b>5</b> | <b>Conclusion</b>                            | <b>24</b> |
| 5.1      | Results Achieved . . . . .                   | 24        |
| 5.2      | Summary of Design Decisions . . . . .        | 24        |
| 5.3      | Limitations . . . . .                        | 26        |
| 5.4      | Future Directions . . . . .                  | 26        |
|          | <b>References</b>                            | <b>26</b> |

# List of Tables

|     |                                                                        |    |
|-----|------------------------------------------------------------------------|----|
| 4.1 | Technology stack per pipeline layer. . . . .                           | 14 |
| 5.1 | Summary of major design decisions and their deciding trade-offs. . . . | 25 |

# List of Figures

|    |                                                                                                                                                |    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Producers and consumers decoupled through a publish-subscribe message queue. . . . .                                                           | 6  |
| 2  | Batch processing versus streaming (event-at-a-time and micro-batching).                                                                        | 7  |
| 3  | Object storage versus the table abstraction built on top of it. . . . .                                                                        | 8  |
| 4  | Overview of an open table format, contrasting copy-on-write and merge-on-read. . . . .                                                         | 8  |
| 5  | Medallion architecture: successive bronze-to-gold refinement layers. . .                                                                       | 9  |
| 6  | Streamify architecture overview . . . . .                                                                                                      | 13 |
| 7  | Kafka internal architecture: 2-node KRaft cluster, 3 partitions per topic, replication factor 2. . . . .                                       | 15 |
| 8  | Spark cluster architecture: driver, master, and worker/executor processes.                                                                     | 16 |
| 9  | Spark Structured Streaming’s micro-batch model: an unbounded stream is sliced into small batches, each run as an ordinary Spark batch job. . . | 17 |
| 10 | Star schema: <code>fact_streams</code> and its five dimension tables. . . . .                                                                  | 20 |
| 11 | dbt DAG showing the dependency between bronze sources, the star schema, and <code>wide_streams</code> . . . . .                                | 20 |
| 12 | Superset results dashboard, built on top of <code>wide_streams</code> and queried through the Spark Thrift Server. . . . .                     | 21 |
| 13 | Grafana Kafka Lag dashboard: consumer-group lag and consumption rate per topic. . . . .                                                        | 23 |
| 14 | Grafana Spark dashboard: streaming/batch job metrics, JVM heap, and per-container CPU/memory. . . . .                                          | 23 |

# Chapter 1

## Introduction

*In chapter 1, the overview, objectives, and goals of the research project are illustrated. The outline of the report is also presented.*

### 1.1 Motivation

Modern data platforms increasingly need to react to events as they happen rather than waiting for the next batch load. A music-streaming service, for example, wants to know within seconds which songs are trending, whether a user’s session just dropped, or whether ingestion is falling behind – questions a traditional overnight ETL job cannot answer. Building and operating such a platform correctly requires more than a single tool: it requires a message queue that can buffer and replay events, a stream-processing engine that can keep up with continuous arrival, a storage layer that supports transactional updates on top of cheap object storage, a transformation layer that turns raw events into trustworthy analytics tables, and a way to observe the whole system’s health while it runs.

This project, *Streamify*, was motivated by the goal of building and demonstrating exactly such an end-to-end pipeline, using synthetic but realistic music-streaming events generated by Eventsim, rather than studying any single one of these components in isolation. Each stage of the pipeline – ingestion, stream processing, lakehouse storage, transformation, and visualization – was chosen and configured to work together as a single reproducible system, so that the trade-offs made at each stage (Chapter 2) could be

evaluated in the context of the whole pipeline rather than as isolated technology choices.

A further motivation was to explore these technologies under realistic resource constraints. Rather than assuming unlimited cloud compute, the project deliberately targets a small, single-machine deployment (Section 4.7), so that decisions such as Kafka cluster sizing and Spark cluster sizing (Section 4.7) had to be made under the same resource pressure a small team or a proof-of-concept deployment would face in practice.

## 1.2 Goals

The goals of this project are:

- **Build a working, end-to-end streaming lakehouse pipeline.** Take synthetic music-streaming events from generation (Eventsim) through ingestion (Kafka), stream processing (Spark Structured Streaming), lakehouse storage (Hudi), transformation (dbt), and visualization (Superset), as a single reproducible system rather than isolated proofs of concept.
- **Demonstrate streaming-specific concerns, not just data movement.** Exercise partitioning, replication, checkpointed exactly-once processing, and consumer-lag monitoring, so the pipeline shows the behaviors that distinguish a streaming design from a periodic batch job (Section 2.4).
- **Justify every major technology choice against concrete alternatives.** For each stage of the pipeline, identify the alternatives that were available, the trade-off that mattered for this project, and why the chosen option won (Chapter 2), instead of treating the stack as a given.
- **Make the whole system observable and reproducible.** Provide Prometheus/Grafana monitoring of Kafka lag and Spark resource usage, and deploy the entire stack with a single Docker Compose command.

## 1.3 Scope

This project covers the full path from synthetic event generation to dashboarding: Eventsim event generation, Kafka ingestion, Spark Structured Streaming processing into Hudi bronze tables, dbt transformation into a star schema and wide table, Superset dashboarding, and Prometheus/Grafana monitoring of the pipeline’s own health (Chapter 4). All of these components are deployed and demonstrated together via Docker Compose on a single machine.

The following are explicitly out of scope:

- **Production security hardening.** Authentication, authorization, network isolation, secrets management, and TLS between services are not addressed; the stack is configured for local development and demonstration, not for exposure to untrusted networks.
- **Multi-region or high-availability production deployment.** The pipeline targets a single-machine, single-region Docker Compose deployment (Section 4.7); cross-region replication, multi-datacenter failover, and elastic autoscaling of the Kafka or Spark clusters are not implemented.
- **Real user data.** All events are generated by Eventsim against the Million Song Dataset’s 10,000-song subset (Chapter 3) and are entirely synthetic; the pipeline has not been run against real production traffic or real user data.

## 1.4 Thesis Structure

The remainder of this report is organized as follows:

- **Chapter 2, Related Work and Background Theory,** first covers the core concepts behind the pipeline – event streaming and pub-sub messaging, Spark Structured Streaming’s micro-batch model, Hudi’s table formats, and data modeling.
- **Chapter 3, Problem Statement and Challenges,** describes the dataset used and the technical challenges of building a fault-tolerant streaming pipeline on a small, shared compute cluster.

- **Chapter 4, Proposed Solution**, presents the system architecture, the implementation workflow, and the cluster/environment setup.
- **Chapter 5, Conclusion**, summarizes the results achieved, current limitations, and directions for future work.

## Chapter 2

# Related Work and Background

This section introduces the general concepts behind a streaming lakehouse – message queues, stream processing engines, open table formats, and dimensional modeling – independent of any one product. Chapter 4 then explains which concrete tool implements each concept in this project, and why that tool was chosen over its alternatives.

### 2.1 Why Real-Time Processing Matters

A modern platform such as a music-streaming service does not produce data in daily batches: it produces a continuous stream of small events – a song played, a page viewed, a login – the instant a user acts. Streamify, the project this report builds, simulates exactly this kind of platform with Eventsim, which generates synthetic “song played”, “page view”, and “authentication” events for a fake music website at a steady, ongoing rate.

In a traditional batch-processing setup, these events would simply accumulate in a source system and be pulled into the warehouse on a fixed schedule – once an hour, once a day. Between runs, every dashboard and every downstream decision is working from data that is, on average, half of that interval out of date; right before the next run, it is a full interval out of date. For a platform that wants to react to what users are doing now – surfacing trending songs, flagging a spike in failed logins, feeding a recommendation engine – that lag is the problem: the data is correct, just always late.

Real-time (streaming) processing closes this gap. Instead of waiting for a scheduled run, each event is picked up and made queryable within seconds to minutes of happening,

so dashboards and downstream models reflect what is happening now rather than what happened last interval. That said, doing this reliably, and not just quickly, is harder than it sounds once the platform has more than one producer and more than one consumer of the same data: events from different partitions can arrive out of order, any component in the chain can crash mid-stream, the same event must not be silently duplicated or dropped on retry, upstream fields will change shape over time, and the storage layer has to serve both a constant stream of small writes and large analytical reads without one starving the other. These are the distributed-systems problems that the rest of this section’s concepts exist to solve: a *message queue* for ordered, fault-tolerant ingestion; a *stream processing engine* for continuous, exactly-once processing; and an *open table format* for a storage layer that supports both frequent streaming writes and fast analytical reads.

## 2.2 Message Queues and Publish-Subscribe Messaging

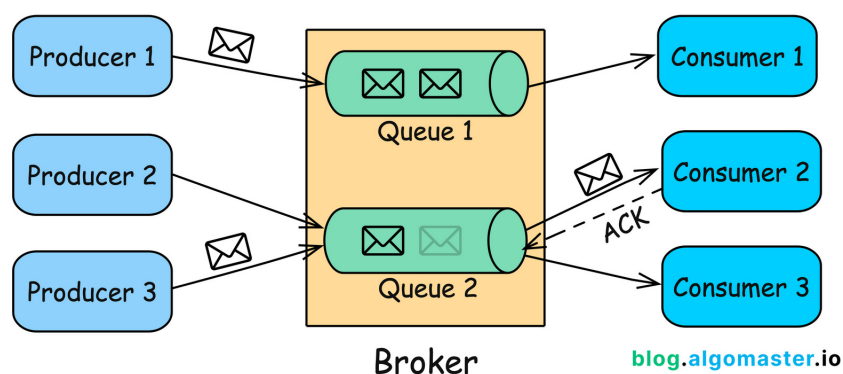


Figure 1: Producers and consumers decoupled through a publish-subscribe message queue.

Event streaming treats data as an unbounded sequence of immutable events (e.g. “user played a song”) rather than as static rows in a table. A *message queue* built for this pattern is typically a log-based, publish-subscribe (pub-sub) broker, shown in Figure 1: producers append events to a durable, ordered log rather than pushing directly to a fixed receiver, and publish them under named *topics*; one or more consumers subscribe to the topics they need and read the log independently, at their own pace. This decouples producers from consumers in both time and number – a producer does not need a con-

sumer to be running or fast enough to keep up, and any number of independent consumers can read the same retained events without the broker deleting them on delivery.

## 2.3 Stream Processing Engines

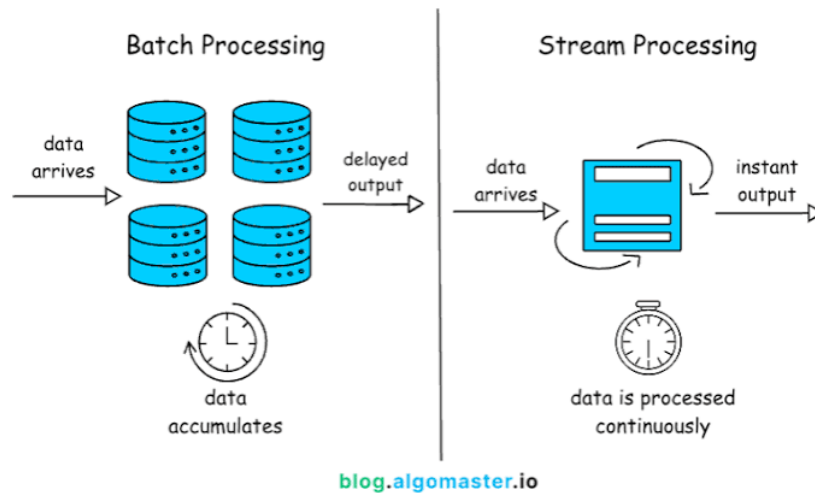


Figure 2: Batch processing versus streaming (event-at-a-time and micro-batching).

A stream processing engine continuously reads from a message queue, applies transformations, and writes results, without waiting for a full batch of data to accumulate first. Figure 2 contrasts this against traditional batch processing. Two execution models are used in practice:

- **Event-at-a-time.** Each event is processed as soon as it arrives, giving the lowest possible per-event latency.
- **Micro-batching.** The engine groups whatever new data has arrived since the last run into a small, discrete batch job on a fixed trigger interval, and processes that batch with the same execution engine used for ordinary batch queries. This reuses a mature batch engine's query planning and fault tolerance, at the cost of end-to-end latency being bounded by the trigger interval rather than being instantaneous.

Either model reads its input positions (offsets) from the upstream message queue and applies the same kind of transformation logic (filtering, aggregation, joins) to produce an output.

## 2.4 Storage layer & Open Table Formats

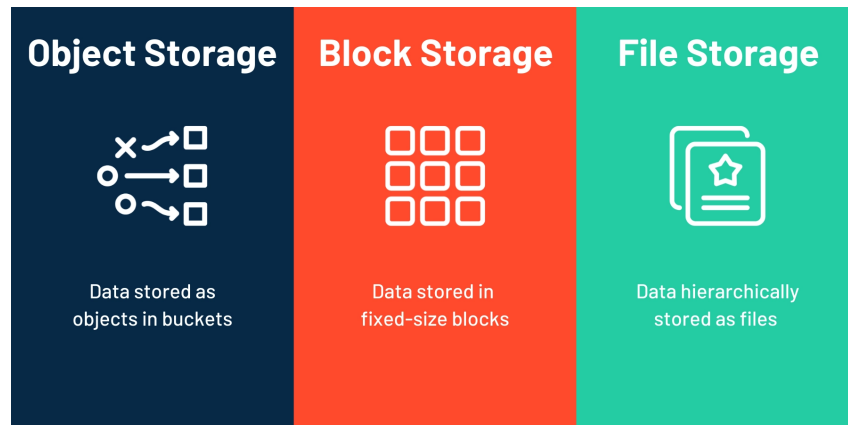


Figure 3: Object storage versus the table abstraction built on top of it.

Object storage (S3, MinIO, HDFS, ...) is a cheap, scalable, and durable way to store large amounts of data, but it is not a database: it does not provide row-level insert/update/delete semantics, ACID transactions, or time-travel queries. An *open table format* adds those capabilities on top of plain files in object storage, so a lake of files behaves like a table rather than an unmanaged pile of Parquet, as shown in Figure 3.

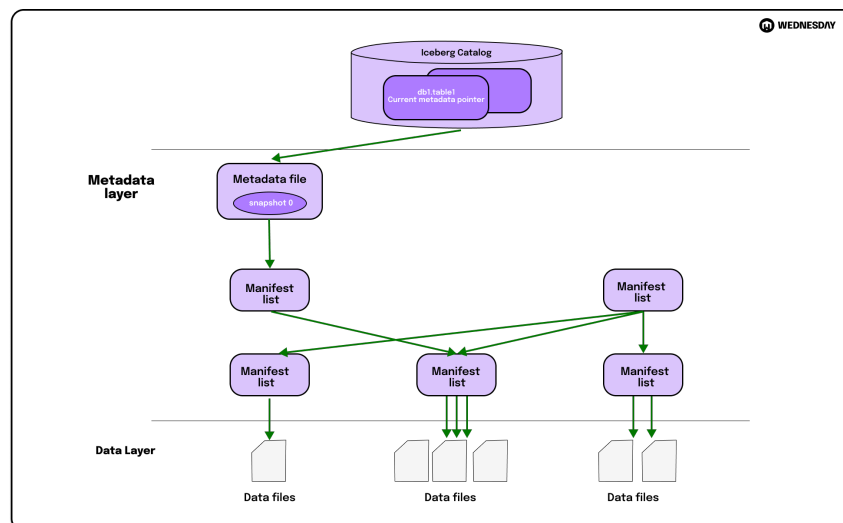


Figure 4: Overview of an open table format, contrasting copy-on-write and merge-on-read.

Open table formats typically support two write strategies that trade off write cost against read cost, as summarized in Figure 4:

- **COPY\_ON\_WRITE (COW).** Every write rewrites the affected data files as new versioned files. Reads are fast (plain columnar files, no merge step), but writes are more expensive because entire files are rewritten on update.
- **MERGE\_ON\_READ (MOR).** Writes are appended to a compact row-based delta log next to the existing base files; the two are merged only at read time (or during a background compaction job). Writes are cheap and low-latency, at the cost of extra work on read (or a periodic compaction step) to materialize the merged view.

MOR suits high-frequency streaming appends, where write latency matters most; COW suits tables that are rebuilt in full on a schedule and then read many times, where read performance matters most.

## 2.5 Medallion Layering



Figure 5: Medallion architecture: successive bronze-to-gold refinement layers.

The medallion architecture, shown in Figure 5, organizes a lakehouse into successive layers of increasing refinement, conventionally named bronze, silver, and gold. *Bronze* tables hold raw or lightly-cleaned data close to its source form, written as-is by the ingestion layer. *Gold* tables hold data that has been modeled and aggregated into the shape consumers (BI tools, dashboards) actually query, typically built by a transformation layer reading from bronze. This project uses the bronze and gold terms in that sense throughout.

## Chapter 3

# Problem Statement and Challenges

### 3.1 Dataset

The pipeline is driven by Eventsim, a synthetic event generator that simulates page-request and listening activity on a fake music-streaming website (the Docker image used is viirya’s fork of the original, unmaintained project). Eventsim draws its song catalog from the Million Song Dataset; this project uses the official 10,000-song subset. The generated events (`listen_events`, `page_view_events`, `auth_events`) look like real user-activity logs but are entirely synthetic, which makes them safe to publish and reproduce without any user-privacy concern.

### 3.2 Technical Challenges

Building a streaming lakehouse pipeline on top of this dataset raised the following challenges.

#### 3.2.1 Streaming Volume and Velocity

Eventsim continuously emits events across three topics, so the pipeline must ingest, decode, and land data in the lakehouse without falling behind the producer. This requires a message queue that can buffer and parallelize consumption (Kafka, partitioned per topic) and a streaming engine that can keep its micro-batch processing time below the

arrival rate.

### **3.2.2 Exactly-once, Fault-tolerant Processing on a 2-node Kafka Cluster**

Kafka runs as a 2-node KRaft cluster (no ZooKeeper), with each topic replicated across both brokers (replication factor 2, 3 partitions per topic). With only two brokers, the cluster must keep serving every partition if either single broker is restarted or fails, while Spark Structured Streaming must resume exactly where it left off after any restart. This project relies on Structured Streaming's checkpointed offsets (persisted to a durable checkpoint volume) as the source of truth for resuming the stream, and on Kafka's per-partition replication across the two brokers for broker-level fault tolerance.

### **3.2.3 Computing and Partitioning on a Small, Shared Spark Cluster**

The Spark Standalone cluster totals only 4 cores across one master and two workers (2 cores / 2 GB each), and those 4 cores are shared between two long-running applications: the Spark Structured Streaming ingestion job (capped at 2 cores) and the Spark Thrift Server that dbt submits its SQL transformations through (also capped at 2 cores). This leaves no spare capacity for ad-hoc Spark SQL at this cluster size, and means both the streaming job's micro-batch partitioning and dbt's batch transformation jobs must fit within their fixed 2-core budget rather than being able to scale out elastically.

### **3.2.4 Dirty Keys Breaking Dimensional Joins**

Eventsim's raw event fields are not clean enough to join on directly. Text fields such as `artist` and `song` arrive with latin1-mojibake encoding, which the streaming job corrects before the bronze tables are written. Even after that correction, the same artist and song names carry stray quote and backslash characters that differ inconsistently between the `listen_events` stream and the artist/song dimension source, so `fact_streams` cannot join on the raw strings: it strips quotes and backslashes from both sides of the join (`REPLACE(REPLACE(artist, '"', ''), '\\', '')`) before matching against `dim_artists` and `dim_songs`. This is a concrete instance of a broader challenge for

the pipeline: dimensional joins are only as reliable as the string normalization applied to their join keys, and that normalization has to be decided and implemented explicitly rather than assumed.

# Chapter 4

## Proposed Solution

### 4.1 Architecture Overview

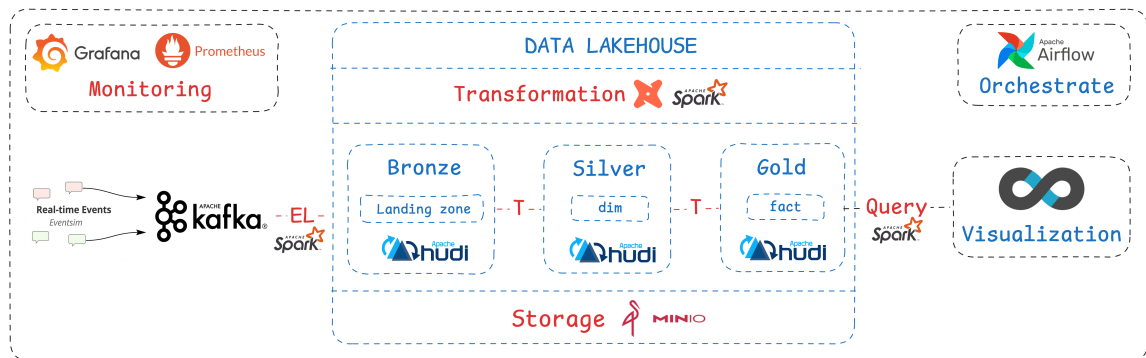


Figure 6: Streamify architecture overview

Figure 6 shows the end-to-end architecture: Eventsim generates events into a Kafka cluster, Spark Structured Streaming consumes and lands them as Hudi bronze tables, dbt transforms the bronze layer into a star schema and a wide table on the same Spark cluster, and Superset visualizes the result through the Spark Thrift Server, alongside a Prometheus/Grafana monitoring stack.

Table 4.1: Technology stack per pipeline layer.

| Layer                  | Tool                                   | Role                                                                                                            |
|------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Data simulation        | Eventsim                               | Generates synthetic music-streaming events based on the Million Song Dataset                                    |
| Message queue          | Apache Kafka 4.3 (KRaft)               | 2-node cluster, replication factor 2, 3 partitions per topic, no ZooKeeper                                      |
| Stream processing      | Spark Structured Streaming 3.5.3       | Reads Kafka, decodes/enriches events, writes bronze tables to Hudi                                              |
| Table format / storage | Apache Hudi                            | MERGE_ON_READ bronze streaming tables, COPY_ON_WRITE gold/seed tables, on MinIO/S3 via the Hadoop S3A connector |
| Compute cluster        | Spark Standalone (1 master, 2 workers) | Shared by the streaming job and dbt (via Thrift), 4 cores total                                                 |
| Transformation         | dbt (dbt-spark, method: thrift)        | SQL-based star schema and wide-table modelling, submitted via                                                   |

The remainder of this chapter walks through the pipeline one layer at a time – ingestion, stream processing, storage, transformation, and serving – followed by how the whole stack is deployed. Each layer section closes with a short note on why that specific tool was chosen, cross-referencing the fuller alternatives-and-trade-offs discussion in Chapter 2.

## 4.2 Ingestion Layer

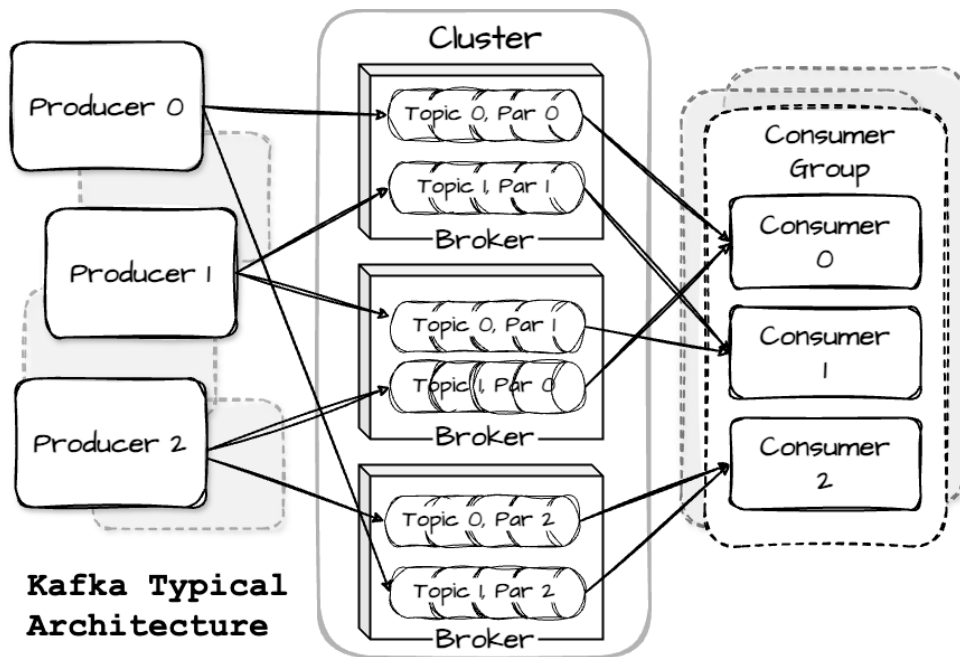


Figure 7: Kafka internal architecture: 2-node KRaft cluster, 3 partitions per topic, replication factor 2.

Eventsim simulates a real-world user base, generating page-view and song-play events that mimic activity on a music-streaming website. Apache Kafka 4.3 runs in KRaft mode (no ZooKeeper) as a 2-node cluster (broker-1, broker-2). Each of the three event topics – `listen_events`, `page_view_events`, `auth_events` – has 3 partitions and replication factor 2, so every partition has a replica on both brokers, as shown in Figure 7.

Because KRaft mode is used, each broker itself also acts as a metadata/controller node, removing the need for a separate ZooKeeper ensemble. For each topic, the 3 partitions are spread across the two brokers, and because replication factor is 2, every partition is fully replicated: one broker holds the leader replica for a given partition and the other

holds the follower replica, and the two brokers swap this role across partitions so that neither broker is leader for every partition of a topic. Producers (Eventsim) and consumers (the Spark Structured Streaming job) always read from and write to the leader replica of a partition; the follower replica only pulls data from the leader to stay in sync, and takes over as leader if its broker fails. This is what gives the cluster broker-level fault tolerance despite running on just two nodes.

**Why Kafka, and why this sizing.** Kafka’s durable, partitioned log and per-consumer-group offsets let Structured Streaming resume ingestion exactly where it left off after a restart, which a direct HTTP push or a traditional queue such as RabbitMQ does not provide as directly. The 2-node, replication-factor-2 sizing is the smallest configuration that gives the cluster any broker-level fault tolerance at all, chosen over a single broker (no fault tolerance) or a larger cluster (more resource cost than this project’s single-machine deployment justifies).

## 4.3 Stream Processing Layer

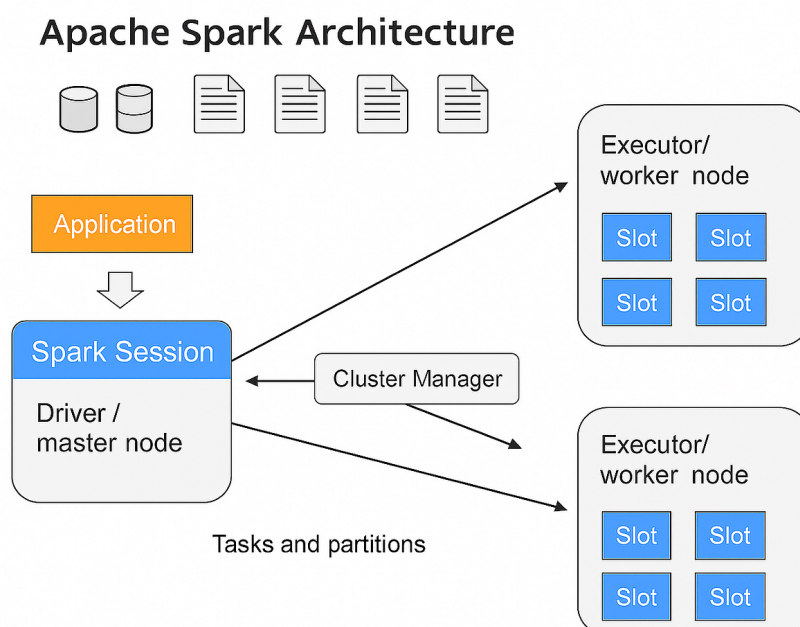


Figure 8: Spark cluster architecture: driver, master, and worker/executor processes.

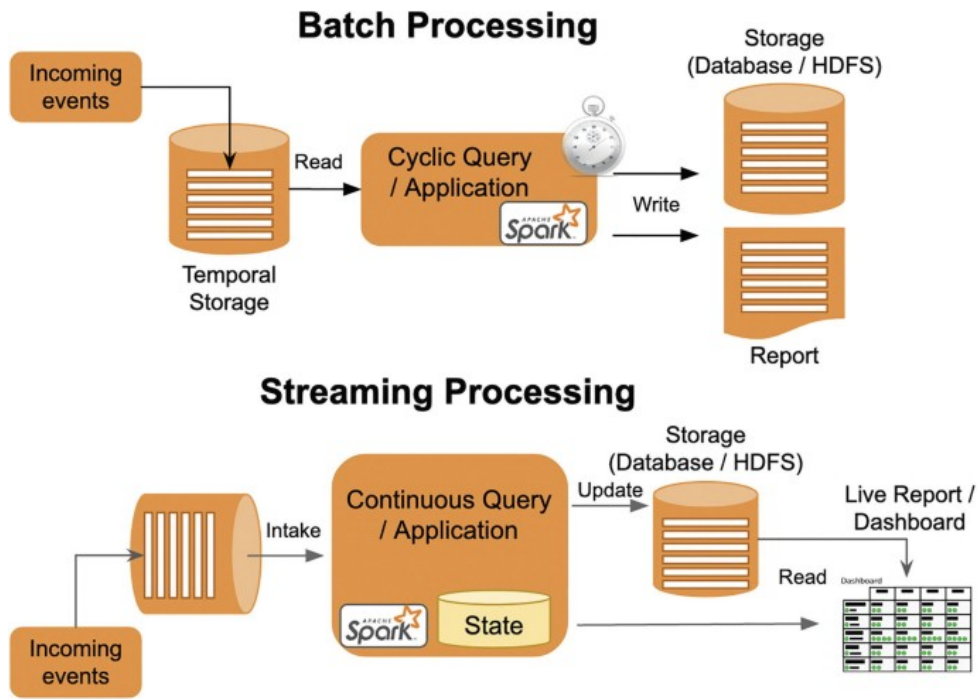


Figure 9: Spark Structured Streaming’s micro-batch model: an unbounded stream is sliced into small batches, each run as an ordinary Spark batch job.

A long-running Spark Structured Streaming job consumes all three Kafka topics, fixes latin1-mojibake on text fields, derives time columns, and appends each micro-batch to bronze Hudi tables (`staging.<topic>`). Figure 8 shows the cluster this job runs on, and Figure 9 shows how Structured Streaming turns an unbounded Kafka stream into the sequence of discrete jobs Spark actually executes.

The streaming job runs as a driver program submitted to the Spark Standalone cluster described in Section 4.7 (1 master, 2 workers, 4 cores total, shared with the Thrift Server dbt submits through). Structured Streaming does not process events one at a time; instead, the driver repeatedly polls the three Kafka topics for new offsets, groups the newly available records into a micro-batch, and schedules that micro-batch as a short-lived Spark job whose tasks run on the worker executors, in the same way a batch job would. Each micro-batch’s tasks read their assigned Kafka partitions, apply the transformations from Listing 4.1, and append the results to the bronze Hudi tables before the driver commits the checkpoint and moves on to the next batch. Because this job and the Thrift Server share the same 4-core cluster, the size and frequency of micro-batches are bounded by whatever cores are not already claimed by the Thrift Server at that moment.

Listing 4.1 shows the core of the job that reads a Kafka topic, decodes it, and derives

the time columns later used to partition the bronze Hudi tables.

Listing 4.1: Building a Hudi bronze stream from a Kafka topic (excerpt from `spark_streaming/jobs/stream_events.py`).

```
1 def build_stream(spark, topic, decode_strings):
2     raw = (
3         spark.readStream.format("kafka")
4         .option("kafka.bootstrap.servers", KAFKA_BROKERS)
5         .option("subscribe", topic)
6         .option("startingOffsets", "earliest")
7         .load()
8     )
9
10    parsed = raw.select(
11        F.from_json(F.col("value").cast("string"), SCHEMAS[topic]).alias
12        "partition",
13        "offset",
14    ).select("data.*", "partition", "offset")
15
16    if decode_strings:
17        parsed = (
18            parsed.withColumn("artist", decode_udf(F.col("artist")))
19            .withColumn("song", decode_udf(F.col("song")))
20        )
21
22    ts_col = F.timestamp_millis(F.col("ts"))
23    return (
24        parsed
25        .withColumn("ts_timestamp", ts_col)
26        .withColumn("year_", F.year(ts_col))
27        .withColumn("month_", F.month(ts_col))
28        .withColumn("day_", F.dayofmonth(ts_col))
29        .withColumn("hour_", F.hour(ts_col))
30        .withColumn("processed_time", F.current_timestamp())
31        .withColumn("_row_key", F.expr("uuid()"))
```

Structured Streaming resumes from a persisted checkpoint volume (`spark-checkpoints`) on restart; each micro-batch also commits its Kafka offsets to a `streamify-<topic>` consumer group purely so lag is visible in Grafana (Spark itself never depends on that commit to resume).

**Why Spark Structured Streaming, and not Flink.** An earlier version of this project used Apache Flink for this job. It was replaced with Spark Structured Streaming so that stream processing runs on the same engine and cluster that dbt already needs for batch transformation (avoiding operating two separate distributed runtimes on a 4-core cluster), and to reuse Hudi's first-class Spark data source support. The trade-off accepted is latency: Flink's event-at-a-time model can achieve lower per-event latency than Spark's micro-batch model.

## 4.4 Storage Layer

Tables are Hudi-managed, backed by MinIO/S3 locally or real S3 in aws mode via the Hadoop S3A connector, with no rebuild needed to switch between the two – only an environment variable (`LAKE_ENV`). There is no separate catalog service: each table is self-describing at its storage path (a `.hoodie/` metadata directory) and is (re-)registered into Spark's catalog idempotently at job startup. Bronze tables use the `MERGE_ON_READ` table type, so fast streaming appends land as delta log files that are periodically compacted into columnar base files; gold and seed tables use `COPY_ON_WRITE`, rebuilt via full overwrite.

**Why Hudi, and why MOR bronze / COW gold.** Hudi's self-describing `.hoodie/` metadata avoids the extra catalog or warehouse service that plain Parquet or Snowflake would require. Bronze uses MOR to keep frequent streaming appends cheap; gold uses COW because it is rebuilt once per run and then read repeatedly by Superset.

## 4.5 Transformation Layer

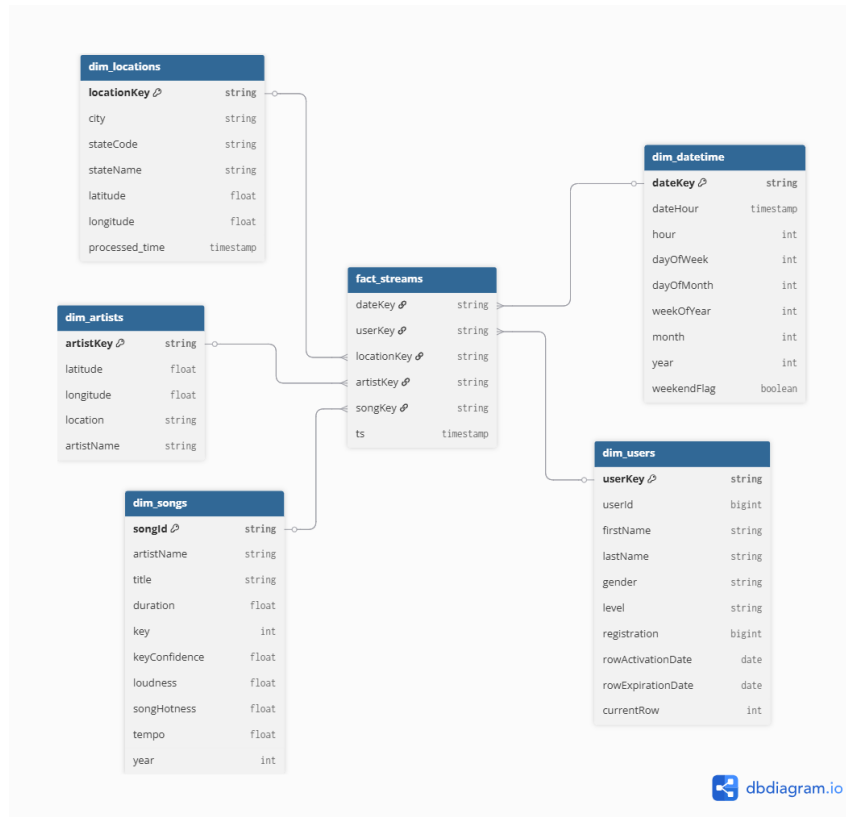


Figure 10: Star schema: fact\_streams and its five dimension tables.



Figure 11: dbt DAG showing the dependency between bronze sources, the star schema, and wide\_streams.

dbt [2] connects to the Spark cluster via the Spark Thrift Server (method: `thrift`) and builds two model layers directly on top of the bronze Hudi tables, materialized as `COPY_ON_WRITE` Hudi tables rebuilt via full overwrite each run. Figure 10 shows the

resulting star schema, and Figure 11 shows the dbt DAG that builds it:

- **Star schema.** One fact table, `fact_streams`, joined to five dimension tables: `dim_users`, `dim_songs`, `dim_artists`, `dim_locations`, and `dim_datetime`. This is the normalized, storage-efficient model (Section 2.4).
- **Wide table (`wide_streams`).** A denormalized table built from the star schema by pre-joining `fact_streams` with all five dimension tables, so that Superset can query a single flat table instead of performing joins at dashboard-query time.

**Why dbt, and why both a star schema and a wide table.** dbt was chosen over a layer of hand-written Spark SQL scripts because it declares model dependencies explicitly (via `ref()`/`source()`), runs a built-in test framework against every model (uniqueness, not-null, referential integrity), and generates documentation from the same source files – properties that matter given the five-dimension join the fact table depends on. Modelling both a star schema and a separate wide table, rather than either alone, keeps normalization and storage efficiency in the star schema while keeping BI query simplicity in wide streams, instead of trading one off against the other.

## 4.6 Serving Layer

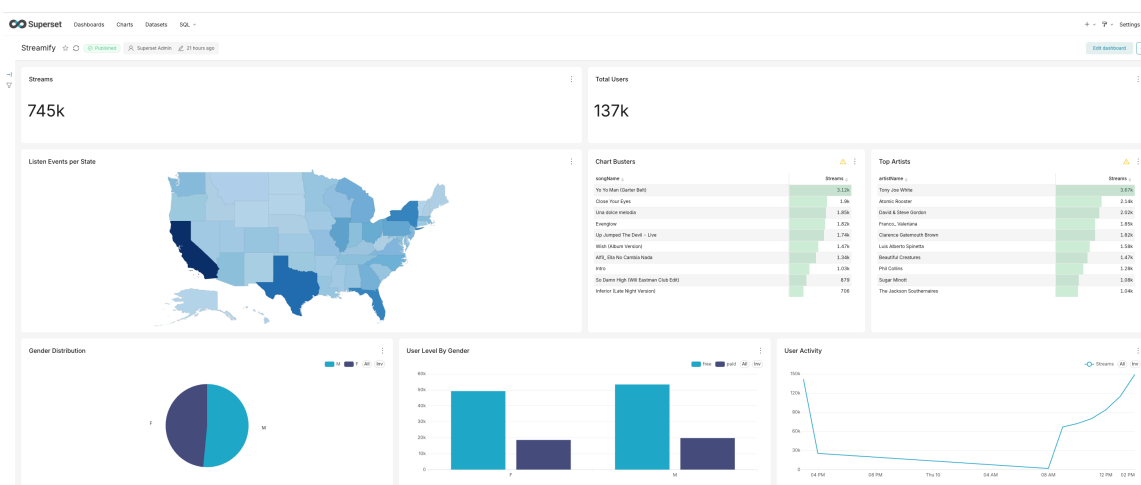


Figure 12: Superset results dashboard, built on top of wide\_streams and queried through the Spark Thrift Server.

Apache Superset [1] connects to the same Spark Thrift Server that dbt submits SQL through, querying the `wide_streams` model to visualize insights through interactive dashboards (`superset/provision_dashboard.py` provisions them via the REST API). Figure 12 shows the resulting dashboard layout.

Prometheus scrapes Kafka consumer-group lag (via `kafka-exporter`), Spark's built-in Prometheus metrics, and per-container stats (via `cAdvisor`); Grafana visualizes all of it across dashboards.

**Why Superset.** Superset ships a native Thrift/HiveServer2-compatible connector that talks directly to the Spark Thrift Server endpoint already used by dbt, needing no extra license or gateway infrastructure, unlike Power BI or Tableau.

## 4.7 Deployment & Environment

The whole stack is deployed with Docker Compose (`docker-compose.yml`), with a `docker-compose.dev.yml` overlay that adds MinIO for local, S3-free development. Spark runs as a Standalone cluster of 1 master and 2 workers, 2 cores / 2 GB each (4 cores total), shared between the streaming ingestion job and the Spark Thrift Server that dbt submits SQL through, each capped at 2 cores.

**Why Spark Standalone at this size.** Spark Standalone was chosen over YARN or Kubernetes because both alternatives add a separate resource-management control plane that is not justified when the entire compute budget is 4 cores across one master and two workers; Standalone instead runs as three containers alongside the rest of the pipeline in the same `docker-compose.yml`. The direct consequence is that the streaming job and the Thrift Server, each capped at a fixed 2 cores, together already consume the entire cluster, leaving no headroom for ad-hoc Spark SQL while both are running.

## 4.8 Monitoring as Qualitative Evidence

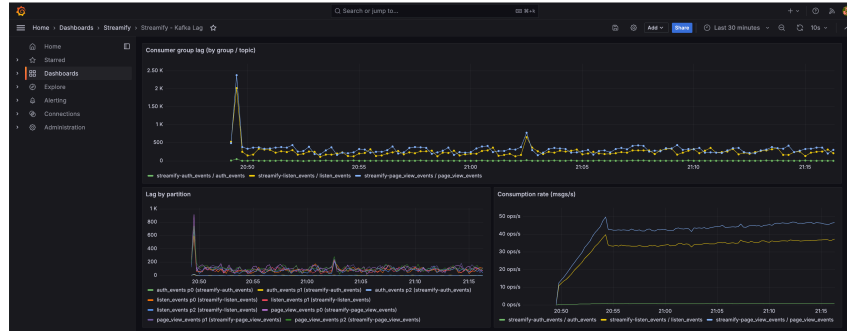


Figure 13: Grafana Kafka Lag dashboard: consumer-group lag and consumption rate per topic.



Figure 14: Grafana Spark dashboard: streaming/batch job metrics, JVM heap, and per-container CPU/memory.

The Grafana dashboards backed by Prometheus, shown in Figures 13 and 14, provide qualitative evidence of the pipeline’s throughput and health under load, rather than a formal benchmark.

# Chapter 5

## Conclusion

### 5.1 Results Achieved

This project delivered a working, end-to-end streaming lakehouse pipeline: Eventsim-generated events flow through a 2-node Kafka cluster into a Spark Structured Streaming job that lands MERGE\_ON\_READ Hudi bronze tables, which dbt transforms (via the Spark Thrift Server) into a COPY\_ON\_WRITE star schema (fact\_streams and five dimension tables) plus a denormalized wide\_streams table. Apache Superset dashboards query that wide table through the same Thrift Server, and a Prometheus/Grafana/cAdvisor stack monitors Kafka consumer lag and Spark resource usage throughout. The entire stack is reproducible with a single Docker Compose command.

### 5.2 Summary of Design Decisions

Table 5.1 summarizes the major technology choices made throughout this project, the alternatives considered for each, and the trade-off that decided the outcome. The ingestion, stream-processing, and storage/table-format choices are discussed in full in Chapter 2; the remaining choices are discussed alongside the layer they belong to in Chapter 4.

Table 5.1: Summary of major design decisions and their deciding trade-offs.

| Decision            | Alternatives considered       | Deciding trade-off                                                                            |
|---------------------|-------------------------------|-----------------------------------------------------------------------------------------------|
| Kafka for ingestion | HTTP/REST push, RabbitMQ      | Durable, re-playable log with per-consumer-group off-sets vs. no built-in re-play (Chapter 4) |
| 2-node KRaft, RF2   | Single broker, larger cluster | Smallest sizing giving any broker fault tol-er-                                               |

## 5.3 Limitations

- **Small, fixed-size cluster.** The Spark Standalone cluster totals only 4 cores, shared between the streaming job and the Thrift Server that dbt uses, leaving no headroom for ad-hoc Spark SQL or additional concurrent workloads.
- **No autoscaling.** Kafka brokers, Spark workers, and other services are fixed at deployment time; the pipeline does not scale resources up or down in response to load.
- **Synthetic dataset.** Eventsim generates realistic-looking but entirely synthetic user activity over a fixed 10,000-song catalog, so the pipeline has not been validated against real production traffic patterns or data-quality issues.

## 5.4 Future Directions

- Replace or augment the synthetic Eventsim data with a real or real-scale streaming dataset to validate the pipeline under realistic data-quality and volume conditions.
- Add autoscaling for the Kafka and Spark clusters (e.g. via a container orchestrator) so cluster capacity can grow with load instead of being fixed at deployment time.
- Extend monitoring and alerting beyond the current Grafana dashboards, e.g. automated alerts on consumer lag thresholds, streaming job failures, or dbt run failures.

# References

- [1] Apache Software Foundation. *Apache Superset Documentation*. 2024. URL: <https://superset.apache.org/docs/intro> (visited on 09/12/2026).
- [2] dbt Labs. *dbt Documentation*. 2024. URL: <https://docs.getdbt.com/> (visited on 09/12/2026).